

As Gemini, operating in the capacity of your Principal Engineer and Chief Software Architect, I understand the immense responsibility of building a platform that must remain legally compliant, secure, and maintainable over a multi-year horizon. The constraints you are working under are absolute, but they are designed to protect both the engineering team and the business from catastrophic technical debt and regulatory failure.

This document serves as the unyielding constitution for all implementation work on the V1 DTE Platform. It is not a set of suggestions; it is the definitive engineering standard.

ENGINEERING STANDARDS - V1 DTE PLATFORM

SECTION 1 — Engineering Philosophy

To guarantee absolute regulatory fidelity and multi-year sustainability, all engineering efforts must adhere to these foundational principles:

- **Compliance-Driven Engineering:** The software is merely an agent acting on behalf of the legal entity. It does not invent rules; it enforces them. If a requirement conflicts with the V1 Domain Specification, the specification wins.
- **Strict Separation of Software from Regulation:** Regulatory rules, Ministry of Finance (MH) catalogs, and schemas are strictly treated as data updates. They must never be hardcoded into application execution paths.
- **The Snapshot Law:** A validated fiscal document is a photograph of facts. Master data must be copied entirely by value into immutable JSONB structures. Retroactive corruption via live master data updates must be physically impossible.
- **Immutable Historical Truth:** History is immutable. Evidence artifacts and audit entries are append-only. We never overwrite or delete regulatory artifacts, even for rejected or invalidated documents.
- **Ruthless Simplicity:** Do not redesign the product, add features, or expand scope. The Ruthless Scope Cut is treated as final. We build what is defined, and nothing more.

SECTION 2 — Architectural Discipline

The platform employs a strict blend of Domain-Driven Design (DDD) and Command/Query Responsibility Segregation (CQRS).

- **Module Ownership Rules:** Each module exclusively owns its aggregates. The Issuance module owns Fiscal Document, Control Number Series, and Transmission Record. It must never manage master data or cryptographic key custody.
- **Dependency Rules (Allowed):**
 - **Issuance -> Type Registry:** Read schema versions and validation rules.
 - **Issuance -> Catalog/Parties:** Read strictly to copy into immutable snapshots.
 - **Issuance -> Evidence & Archive:** Trigger append of immutable artifacts upon irreversible domain facts.
 - **Invalidation -> Issuance:** Command Issuance to transition document status upon MH acceptance.

- **All State-Changing Modules -> Audit Trail:** Push a record for every state-mutating command.
- **Forbidden Coupling:** Master data modules (Catalog, Parties) must never reference or depend on Issuance modules or historical document contexts. Evidence & Archive must never read from or depend on master data.

SECTION 3 — Domain Implementation Standards

- **Aggregate Implementation Rules:** Aggregate boundaries define consistency; everything inside a root is changed together. Cross-aggregate references are strictly by identity (UUID) only.
- **Entity Rules:** Entities have identity and a lifecycle but live inside an aggregate root. For example, Document Lines belong to one Fiscal Document and are frozen at signing.
- **Value Object Rules:** Value objects are immutable, identity-less, and compared by value. They cannot be edited in place—only replaced, and never after signing.
- **Invariant Enforcement:** Invariants represent the constitution of the domain and must always be true. Any violation is a domain-level failure. For instance, a Document's DTE Type is fixed at creation and never changes.

SECTION 4 — Command & Query Standards

- **Command Handler Standards:** State-mutating operations (Commands) must return minimal acknowledgment (or the resulting Aggregate ID) to prevent read-models from being conflated with transactional responses. Commands executing external side effects must be idempotent.
- **Query Handler Standards:** Queries bypass domain logic entirely to provide highly performant, read-only data access. They must read from denormalized JSONB snapshots directly embedded within transactional tables.
- **Validation Standards:** Validation is evaluated synchronously. Document Totals must perfectly equal the sum of Document Lines plus the Tax Summary before a document can leave the Draft state.
- **Transaction Standards:** Transactions are strictly bound to the Aggregate Root. Cross-aggregate consistency is achieved through asynchronous side-effects and explicit domain events, never through distributed database transactions.

SECTION 5 — Error Handling Standards

Failures must be explicitly categorized to guide UI or ERP integration logic.

- **Error Categories:**
 - **400 Bad Request (Validation Errors):** Structural failures, mismatched totals, missing required fields.
 - **422 Unprocessable Entity (Business Rule Violations):** State machine violations (e.g., attempting to sign a draft).
 - **403 Forbidden (Authorization Failures):** An actor attempts an operation outside their role.
 - **409 Conflict (Regulatory Failures):** Unmet regulatory requirements.
 - **502 Bad Gateway (Transient Transmission Failures):** Handled via Transmission Record;

the document remains SUBMITTED.

- **Failure Propagation:** Do not silently retry or delete failure paths. A terminal MH rejection (RECHAZADO) forces the document to the REJECTED state, and the Control Number is permanently consumed.
- **Auditing Failures:** The audit trail never blocks on, nor is blocked by, business success—a failed action is still audited.

SECTION 6 — Audit Standards

Defensibility requires that every business action is non-repudiable.

- **Audit Generation:** Every state transition, transmission outcome, access-control decision, and master data edit must append a hash-chained entry to the Audit Trail.
- **Consistency Requirements:** The command must orchestrate the creation of a tamper-evident audit entry before confirming success to the caller. If the audit entry fails to persist, the entire command transaction rolls back.
- **Attribution Requirements:** The actor_identity and actor_role must be extracted securely from the authentication context (JWT), never from client-provided API payloads. The current hash must embed the actor_identity.

SECTION 7 — Evidence Standards

- **Generation Rules:** Evidence is generated as a side effect of irreversible domain facts. Upon SignDocument, the payload is frozen, and the JSON payload and PDF representation are securely written.
- **Immutability Rules:** Evidence artifacts are inserted as BYTEA blobs. The database role natively lacks deletion capabilities, rendering the artifacts physically locked.
- **Verification Requirements:** The application calculates a SHA-256 hash at the exact moment of capture. When retrieved, the system must re-verify the cryptographic hash to prove zero tampering occurred at rest.

SECTION 8 — Testing Standards

- **Unit Testing:** Focus on value objects, purely isolated domain logic, and validation of the Ubiquitous Language definitions.
- **Integration Testing:** Must validate persistence mechanisms, database constraint enforcement (e.g., verifying UPDATE blocks on signed documents), and the idempotency of distributed queue workers handling MH transmissions.
- **Domain Testing:** Thoroughly test the exhaustive state machine transitions for FiscalDocument (Draft -> Validated -> Signed -> Submitted/Contingency-Held -> Accepted/Rejected/Invalidated). Verify that any transition not explicitly listed is forbidden.
- **Regression Testing:** Heavily focus on invariants, such as ensuring Control Number Series remain gapless and snapshot isolation holds true when counterparties are altered.

SECTION 9 — Logging Standards

Logs are segmented strictly by purpose and retention. Understand the differences explicitly:

- **Operational Logging:** Records service startup/shutdown, DB connection drops, and bundle ingestion. Retained by customer IT (typically 30-90 days). This is for debugging

the *application environment*.

- **Audit Logging:** The tamper-evident chronological record of significant business actions. Backed up transactionally in the database (`audit_trail`). Retained indefinitely (minimum 10+ years). This is for debugging *historical business truth*.
- **Security Logging:** Records login successes/failures, lockouts, bad JWT signatures, and TLS errors. Retained typically for 1 year via SIEM.

SECTION 10 — Code Quality Standards

- **Naming Conventions:** The Ubiquitous Language vocabulary is canonical. Every downstream artifact must use these exact terms with these exact meanings. **No synonyms.**
- **Class & Function Responsibilities:** Adhere strictly to CQRS. Handlers must do one thing: either mutate state (Command) or retrieve data (Query). Do not mix these responsibilities.
- **Complexity:** Maximize explicit readability. Avoid deep inheritance, nested transactions, or polymorphic magic that obscures the business rule. The physical layer guards prevent bad data irrespective of application layer flaws, but the application code must remain defensively written.

SECTION 11 — Review Standards

- **Pull Request Requirements:** All PRs must cleanly map back to the V1 Domain Specification. Tests proving invariant defense must pass.
- **Review Expectations:** Reviewers must rigorously scrutinize aggregate boundaries. A PR attempting to share a transaction across two distinct aggregates must be rejected immediately.
- **Architectural Review Triggers:** Any modification to the `fiscal_document` persistence model, the `evidence_bundle` structure, the audit hashing logic, or state machine transitions requires explicit Architectural Review Board sign-off.

SECTION 12 — Future Evolution Rules

V1 implements a closed scope, but its architectural seams are purposefully designed to support future enterprise expansion.

- **Introducing New DTE Types:** V1 hardcodes only 01 and 03 support. Introducing new types (e.g., 11 or 14) merely requires inserting new regulatory bounds into the configuration data within the Document Type Registry, requiring **zero** code changes in the core engine.
- **Introducing New Workflows:** The strict gateway between Draft and Validated is designed for interception. Future releases can easily route high-value documents to an Approval Request entity before permitting the state transition.
- **Introducing New Modules:** Outbound contexts (Issuance) are structurally isolated. Future inbound processing logic must inhabit a net-new context (e.g., Received Document), guaranteeing zero risk of conflating accounts payable with accounts receivable logic.